

비동기 처리와 에러 핸들링

추론 한 건이 서버 전체를 멈추지 않게 — 겹치고, 떼어내고, 잡고, 기록한다

오늘의 한 문장 — 기다림은 겹치고, 무거운 추론은 옆방(스레드풀)으로 떼어내고, 예외는 한곳에서 잡고, 모든 일은 로그로 남깁니다. Day 2의 친절한 서버가, 여러 명이 동시에 불러도 죽지 않는 서버가 됩니다.

오늘 끝내면 할 수 있는 것

네 가지면 충분합니다.

01

왜 멈추는지 안다

3초 걸리는 추론에 세 명이 동시에 오면 마지막 사람이 9초를 기다리는 이유 — '한 명뿐인 일꾼(이벤트 루프)'이라는 구조로 설명할 수 있다.

02

기다림을 겹친다

한 작업이 기다리는 틈에 다른 작업을 진행하는 비동기의 원리를 — 그림 한 장으로 이해한다.

03

무거운 계산은 떼어낸다

추론을 '옆방 일꾼'에게 맡기는 한 줄(`run_in_executor`) — 추론이 도는 중에도 '살아있니?'(헬스체크)에 즉시 답하는 서버를 만든다.

04

터져도 죽지 않게 한다

예외를 한곳에서 잡고, 모든 요청을 로그로 남겨 — 서버는 살아있고, 운영자는 원인을 알고, 사용자에게 안전한 메시지만 간다.

한 명에겐 완벽했던 서버, 셋이 부르면 9초?

Day 2의 API는 혼자 쓸 땐 흠잡을 데가 없었습니다. 그런데 **동시에 세 명이 부르면** — 마지막 사람은 9초를 기다립니다.

먼저 그림으로 감을 잡아 봅시다. 추론(모델이 답을 내는 계산)이 한 건에 3초 걸리는 서버가 있습니다. 요청 A·B·C가 **같은 순간**에 도착하면 — A는 3초 만에 받지만, B는 6초, C는 9초를 기다립니다. 열 명이면 마지막 사람은 **30초**예요.

서버가 느려서가 아닙니다. CPU는 놀지 않습니다. 문제는 **구조**예요. 요청을 받아 처리하는 '일꾼'이 사실상 한 명이라, 앞사람 일이 끝나야 뒷사람을 시작합니다. 줄을 선 거죠.

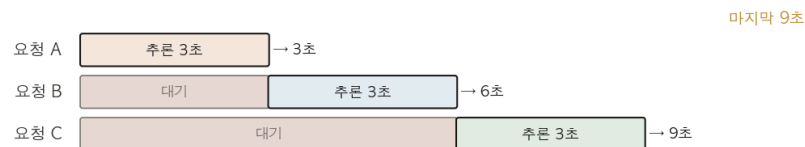
더 무서운 건 '침묵'입니다. 추론이 도는 동안 이 서버는 "살아있니?"라는 점검(헬스체크) 신호에도 답을 못 합니다. 관리 시스템은 응답 없는 서버를 **죽었다고** 판단해 빼버리죠 — 멀쩡한 서버가 쫓겨납니다.

터지는 문제도 같이 옵니다. 사용자가 많아지면 예상 못 한 오류도 늘어납니다. 대비가 없으면 내부 오류 내용이 그대로 노출되고, 기록이 없어 원인도 못 찾고, 최악엔 서버가 통째로 내려갑니다.

오늘의 지도. ① 왜 멈추는지 이해하고 → ② 기다림을 겹치고 → ③ 무거운 추론은 옆방으로 떼어내고 → ④ 예외를 잡고 기록합니다. 코드는 핵심 두 군데만 보고, 나머지는 그림으로 따라갑니다.

동기는 줄을 세우고, 비동기는 틈을 쓴다

동기 (Synchronous) — 한 번에 하나씩



비동기 (Asynchronous) — 대기 시간을 활용



추론 3초 · 세 요청이 동시에 도착 — 동기는 마지막이 9초, 비동기는 셋 다 3초.

01 그림 한 장이면 끝

위(동기)는 막대가 옆으로 줄줄이 이어집니다. A 다음 B, B 다음 C — 그래서 9초. 아래 (비동기)는 막대가 겹쳐서 같이 흐릅니다 — 그래서 셋 다 3초. 시간 축(0→9초)만 봐도 차이가 보입니다.

02 카페로 옮겨 보면

바리스타가 한 명입니다. 동기는 A의 음료가 완성돼야 B 주문을 받아요. 비동기는 A의 머신이 도는 그 틈에 B 주문을 받고 C 블렌더를 컵니다 — 같은 한 명이 9분 일을 3분에 끝냅니다.

03 바뀐 건 '전략'뿐

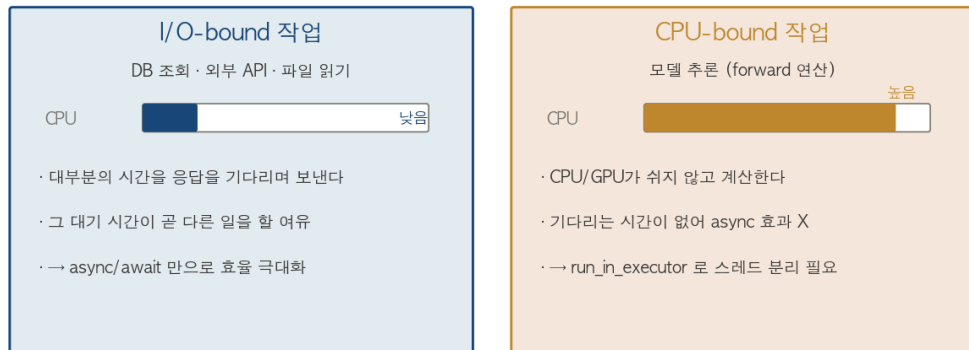
서버를 늘리지도, CPU를 키우지도 않았습니다. 기다리는 틈을 버리지 않는다 — 이 전략 하나로 세 요청이 거의 같이 끝납니다.

04 단, 조건이 있다

이 마법의 재료는 '기다림'입니다. 머신이 도는 틈이 있어야 틈 일을 하죠. 그런데 모델 추론에는 그 틈이 있을까요? — 다음 장에서 확인합니다.

그런데 모델 추론에는 '기다림'이 없다

서버의 일은 크게 둘로 나뉩니다. **I/O-bound**(입출력 중심)는 DB·외부 API·파일처럼 시간 대부분을 **응답을 기다리며** 보내고, **CPU-bound**(계산 중심)는 모델 추론처럼 **쉬지 않고 계산**합니다. 비동기의 재료는 '기다림'인데 — 추론에는 그 재료가 없습니다.



01 I/O-bound — 비동기의 홀그라운드

DB 조회 0.5초의 대부분은 **그냥 기다리는 시간**입니다. 그 틈에 다른 요청을 처리하면 같은 서버로 몇 배를 받아냅니다. 카페의 '머신 도는 시간'이 바로 이것이죠.

02 CPU-bound — 양보할 틈이 없다

추론 3초 동안 CPU/GPU는 계산을 **쉬 없이** 돌립니다. "기다리는 동안 댄 일"을 하고 싶어도 기다림 자체가 없어요 — 끼어들 틈이 없습니다.

03 그래서 추론은 특수하다

엔드포인트를 비동기로 바꿔도 추론은 1초도 빨라지지 않습니다. 오히려 잘못 쓰면 서버 전체가 멈춰요 — 다음 장에서 그 장면을 직접 봅니다.

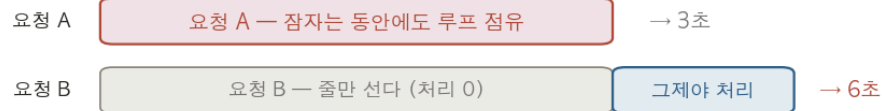
04 해법 미리보기 — '떼어내기'

추론을 '옆방'(별도 일꾼)으로 보내는 방법(§4)이 답입니다. 계산 중심 작업을 비동기 세계로 데려오는 다리예요.

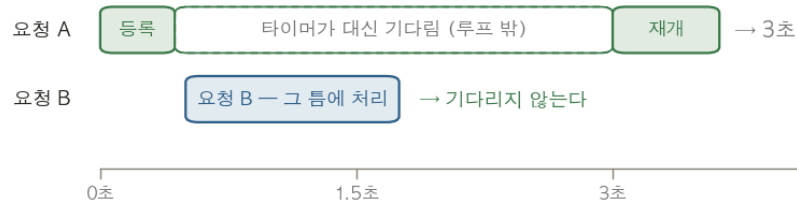
FastAPI의 일꾼은 사실상 '한 명'이다

이벤트 루프는 FastAPI 안에서 할 일 목록을 끝없이 돌며 처리하는 **단 하나의 일꾼**입니다. 수십 명을 동시에 상대하는 비결은 — 한 작업이 '기다림'에 들어가면 잽싸게 다음 작업으로 갈아타는 것. 단, 한 명이라 **누군가 양보 없이 붙들면 전부 멈춥니다**.

`time.sleep(3)` — 루프를 붙들고 잔다



`await asyncio.sleep(3)` — “3초 뒤 깨워줘” 등록하고 비켜준다



01 왜 '한 명'인데 빠를까

손님 여럿을 받는 카운터 직원 한 명을 떠올려 보세요. 주문(요청)을 받고, 머신이 도는 동안 다음 손님을 받습니다. 쉬는 틈을 메우니 한 명으로도 여럿을 상대해요.

02 붙들면 전부 멈춘다

한 손님이 카운터를 3초간 독차지하면, 그 사이 다른 손님도 헬스체크도 모두 대기합니다. '한 명뿐'이라는 게 강점이자 약점이에요.

03 그래서 '어디서 도느냐'가 운명

같은 추론이라도 일꾼(루프) '위'에서 돌면 루프가 멈추고, '옆방'에서 돌면 루프는 자유입니다. 다음 장에서 이 갈림길을 코드로 딱 한 번 봅시다.

04 오늘의 목표

무거운 일은 옆방으로 보내 **루프를 늘 비워 두는 것**. 그래야 헬스체크 같은 가벼운 요청이 언제나 즉시 답합니다.

같은 추론, 어디서 돌리느냐가 갈림길

두 함수는 거의 똑같아 보입니다. 차이는 `async` 한 단어뿐 — 그런데 결과는 정반대입니다. 일반 `def` 는 FastAPI가 알아서 옆방(스레드)으로 보내고, `async def` 안의 무거운 계산은 일꾼(루프) 위에서 직접 돌아 모두를 멈춥니다.

```
# ① 일반 def — FastAPI가 알아서 '옆방 일꾼(스레드)'에게 보낸다
@app.post("/predict")
def predict(req):
    return model(tensor)      # 옆방에서 3초 — 루프는 자유, 다른 요청 OK

# ② async def 안에서 무거운 계산 — 오늘의 함정!
@app.post("/predict")
async def predict(req):
    return model(tensor)      # 루프 '위'에서 3초 — 그동안 루프 정지

@app.get("/health")
async def health():
    return {"status": "ok"}    # ②가 도는 3초 동안 이 응답도 막힌다
```

01 ① 일반 def — 자동으로 옆방

FastAPI가 `def` 핸들러를 **별도 스레드**에서 실행합니다. 추론 3초 동안에도 루프는 비어 있어 다른 요청·헬스체크를 계속 받아요.

02 ② async def + 무거운 계산 — 함정

핸들러가 루프 **위에서 직접** 3초를 듭니다. 그동안 새 요청도 `/health` 도 전부 대기 — 동시성을 얻으려다 오히려 모두 잃는 조합입니다.

03 핵심은 한 단어가 아니다

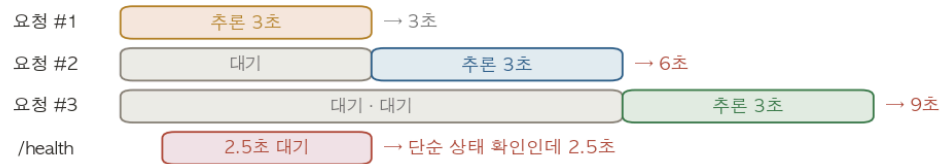
`async` 를 붙였는지가 아니라, **무거운 계산이 루프를 붙드는지**가 본질입니다. 같은 함정을 다음 장 실험에서 시간으로 직접 봅니다.

04 그럼 def만 쓰면 되나?

대부분 동작합니다. 하지만 옆방 일꾼의 수를 **내가 정할 수 없어요**. 더 나은 길은 §4에서 — 먼저 '옆방 일꾼'이 뭔지부터 정리합니다.

일부러 멈춰 봤다 — 헬스체크까지 같이 멈춘다

blocking 서버 (async def + time.sleep) — 한 건이 전부를 세운다



스레드풀 서버 (일반 def / run_in_executor) — 겹쳐 돌고, 맥박은 즉시



01 실험 방법

진짜 모델 대신 '3초 기다리기'로 주문 시간을 정확히 통제했습니다. 함정 버전 (`async def` 위에서 실행)과 옆방 버전, 두 서버에 요청 3개를 동시에 던집니다.

02 함정 버전 — 3·6·9초

요청 #1이 3초, #2가 6초, #3이 9초. **완전한 순차 실행** — §1.1의 시나리오가 그대로 재현됐어요.

03 헬스체크가 2.5초 지연

주문 시작 0.5초 뒤 '살아있니?'를 물었더니 **2.5초** 만에 답합니다. 가장 가벼운 신호조차 줄을 선 거예요 — 일꾼이 붙들려 있으니깐요.

04 옆방 버전 — 모두 3초, 맥박 즉시

같은 3초인데 옆방으로 보낸 버전은 셋 다 3초 안팎, 헬스체크는 **0.0초**. 코드가 아니라 **돌아가는 자리**가 운명을 갈랐습니다.

주문을 3초로 흉내 낸 실험 — 위는 멈추는(blocking) 서버, 아래는 옆방에 보낸 서버.

맥박이 멈춘 서버는, 멀쩡해도 퇴출된다

헬스체크가 2.5초 늦는다는 건 '조금 느리네'가 아닙니다 — 운영 환경에선 장애의 시작 신호입니다.

헬스체크는 서버의 맥박입니다. 여러 서버에 트래픽을 나눠 주는 장치(로드밸런서)는 몇 초마다 `/health` 를 찔러 "살아있나?"를 확인합니다. 제때 답하면 정상, 임계 시간을 넘기면 비정상으로 봅니다 — 판단 기준은 이 응답뿐이에요.

불들린 서버는 맥박을 멈춥니다. 추론이 일꾼(루프)을 붙들면 헬스체크도 줄에서 대기합니다. 서버는 **멀쩡히 일하는 중인데** 답을 못 하는 거죠 — 일하느라 전화를 못 받는 직원과 같습니다.

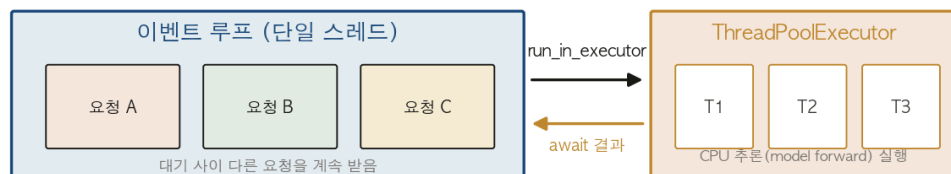
관리 장치는 사정을 모릅니다. 무응답이 이어지면 "죽었다"고 판단하고 그 서버를 트래픽에서 빼버립니다. 가장 열심히 일하던 서버가 **그 일 때문에** 쫓겨나는 역설이에요.

그리고 도미노로 번집니다. 한 대가 빠지면 남은 서버로 부하가 몰리고, 그 서버도 추론에 막혀 헬스체크를 놓치고, 또 빠지고... 멀쩡한 서버들이 차례로 사라지는 전면 장애가 됩니다.

결론 — 일꾼은 늘 비워 뒀야 합니다. 무거운 일은 옆방으로 내보내고, 가벼운 요청은 **언제나 즉시** 답하게 만드는 것. 이것이 다음 섹션의 목표입니다.

'옆방 일꾼들' = 스레드풀

스레드는 일을 동시에 진행하는 '일손' 하나, 스레드풀은 그런 일손을 **미리 정해진 수만큼** 모아 둔 대기조입니다. 무거운 일이 오면 카운터(루프)가 대기조의 일손에게 넘기고 자기는 곧장 다음 손님을 받습니다 — 이 '옆방'이 비동기 추론의 핵심 무대예요.



핵심: 무거운 동기 작업을 이벤트 루프 밖으로 던진다

1. `loop.run_in_executor(executor, model, tensor)` 로 추론을 별도 스레드에 위임
2. 이벤트 루프는 막히지 않고 그 사이 다른 요청-웹스체크를 처리
3. 추론이 끝나면 `await` 가 결과를 돌려받아 응답을 완성

01 왜 '미리.정해진 수'일까

요청마다 일손을 새로 뽑으면 채용 비용이 큼니다. 그래서 처음에 N명을 만들어 두고 재활용해요. 이 N이 곧 '풀 크기'입니다.

02 FastAPI엔 이미 기본 대기조가 있다

일반 `def` 핸들러는 FastAPI가 만들어 둔 **기본 스레드풀**로 자동으로 갑니다. 이 기본 대기조의 크기가 보통 **40명**이에요 — 다음에 나올 숫자의 정체입니다.

03 그런데 그 40을 내가 못 만진다

기본 대기조는 프레임워크 소관이라 크기를 직접 조절할 수 없고, 모든 `def` 가 **함께 나뉘** 씁니다. GPU라서 1~2로 줄이고 싶어도 손댈 수 없죠.

04 그래서 '전용 대기조'를 만든다

추론만 쓰는 스레드풀을 직접 만들어 크기를 정하고, 추론 부하를 다른 일과 분리합니다 — 그 방법이 바로 다음 장입니다.

무거운 추론만 '옆방'으로 — `run_in_executor`

Day 2의 서버에서 **추론 호출 한 줄만** 바꿉니다. 검증·전처리·응답은 그대로예요. 핵심은 `run_in_executor` — "이 함수를 옆방(스레드풀)에서 돌리고, 끝나면 결과를 받아 와"라는 뜻입니다.

```
# 추론 전용 일꾼 4명을 미리 만들어 둔다 (= 스레드풀)
pool = ThreadPoolExecutor(max_workers=4)

@app.post("/predict")
async def predict(req):
    tensor = preprocess(req.pixels)          # 가벼운 일은 루프에서 바로
    loop = asyncio.get_event_loop()
    result = await loop.run_in_executor(      # 무거운 추론만 옆방으로 위임
        pool, model, tensor)                 # pool에게 model(tensor) 시킴
    return result                             # 다 되면 await가 결과를 회수
```

01 전용 대기조를 만든다

`ThreadPoolExecutor(max_workers=4)` — 추론만 쓰는 일꾼 4명을 서버 시작 때 준비합니다. 크기를 **내가** 정하는 게 기본 풀과의 차이예요.

02 가벼운 건 루프, 무거운 건 옆방

픽셀→텐서 변환은 수 ms라 루프에서 바로 합니다. 수백 ms~수 초인 추론만 옆방으로 — **전부**가 아니라 **무거운 것만** 떼어냅니다.

03 던지고 → 딴 일 → 받아오기

`await run_in_executor(...)` 는 추론을 옆방에 던진 뒤 루프를 풀어 줍니다. 그 3초 동안 루프는 새 요청·헬스체크를 처리하고, 추론이 끝나면 `await` 가 결과를 회수해요.

04 '한 번에 하나' 아닌가요?

순수 파이썬 계산은 동시에 하나만 돕니다. 하지만 PyTorch·NumPy의 무거운 연산은 내부에서 그 제약을 잠시 풀어 줘서, 추론은 옆방 일꾼들끼리 실제로 겹쳐 돕니다.

그래서 일손은 몇 명이 적당할까

무작정 늘리면 빨라질 것 같지만 — 정답은 **하드웨어와 작업 성격**이 정합니다.

≈코어수

CPU 추론

4코어면 4명. 계산할 손(코어)은 4개인데 일손만 늘리면, 손 바뀌 주는 낭비 탓에 오히려 느려집니다. `os.cpu_count()`로 코어 수를 확인하세요.

1~2

GPU 추론

GPU는 **그 자체로** 엄청난 병렬 계산기입니다. 일손을 늘려도 GPU 앞에 줄만 길어지고 메모리 부족 위험만 커져요. 1~2명이면 충분합니다.

40

기본 대기조

§4.0에서 본 그 숫자 — 일반 `def`가 쓰는 FastAPI 기본 풀의 크기입니다. 충분해 보여도 **직접 조절이 안 되고** 모든 `def`가 공유하죠. 그래서 전용 풀을 만든 겁니다.

일반 def도 되는데, 굳이? — 차이는 '제어권'

세 버전을 동시 3요청으로 재 보면 — 함정 버전만 9초, 일반 def와 전용 풀은 둘 다 3초. 시간이 같다면 왜 전용 풀일까요?

간단한 경우

일반 def

코드가 가장 단순

함수를 `def` 로 선언만 하면 FastAPI가 알아서 기본 대기조로 보냅니다. 작은 프로젝트라면 이걸로 충분해요.

크기 제어 불가

기본 대기조 크기(40)·동작은 프레임워크 소관입니다. GPU라서 1~2로 줄이고 싶어도 손댈 수 없어요.

대기조를 모두와 공유

모든 `def` 가 **같은 대기조**를 씁니다. 추론이 대기조를 점령하면 파일 다운로드 같은 다른 작업까지 밀려요.

이 과정의 표준

전용 풀 + `run_in_executor`

의도가 코드에 보인다

"이 추론은 무거우니 전용 대기조로 보낸다"는 결정이 명시적입니다. 한번 익히면 실무에서도 그대로 쓰는 패턴이에요.

크기를 직접 결정

일손 수를 하드웨어에 맞춥니다 — CPU는 코어 수, GPU는 1~2. 앞 장의 가이드를 그대로 적용하죠.

추론 부하를 격리

추론 전용 대기조가 따로라, 추론이 몰려도 다른 기능은 영향이 없습니다.

검증을 통과한 입력도, 서버를 터뜨린다

Day 2의 입력 검증은 **잘못된 입력**을 문 앞에서 막았습니다. 그런데 입력이 멀쩡해도 — **서버 안**에서는 언제든지 터질 수 있어요.

내부 사고는 검증이 못 막습니다. 모델 파일이 손상돼 추론이 깨지고, GPU 메모리가 바닥나고, 멀쩡해 보이던 이미지가 디코딩에서 깨지는 일 — 전부 입력 검증을 **통과한 뒤**에 일어납니다.

대비가 없으면 두 가지가 생깁니다. 하나는 **보안** — 내부 오류 내용(어느 파일 몇 번째 줄에서 터졌는지)이 사용자에게 그대로 노출돼 공격의 단서를 줍니다. 또 하나는 **운영** — 기록이 없으면 "어제 왜 죽었지?"에 답할 단서가 0이에요.

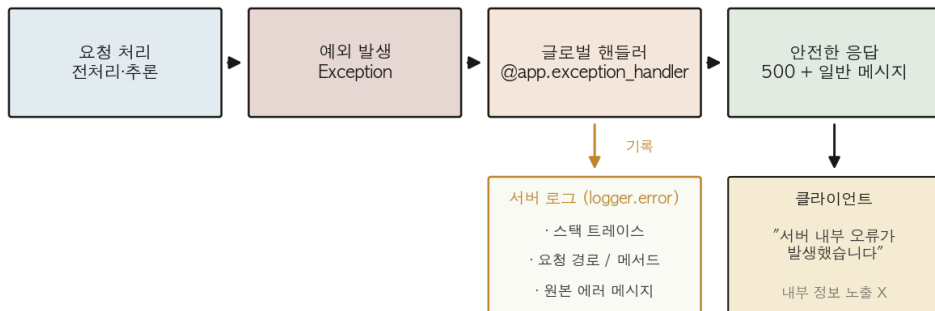
최악은 서버가 죽는 것입니다. 처리되지 않은 오류가 끝까지 올라가면 서버 프로세스가 내려갈 수 있습니다. 한 사람의 이상한 요청 하나가, 나머지 모든 정상 사용자의 서비스를 끊는 셈이죠.

핸들러마다 try/except 도배는 답이 아닙니다. 엔드포인트가 10개면 10번 반복하고, 새로 만들 때 한 번 빠뜨리면 거기서 사고가 납니다. 해법은 반대 방향 — **못 잡은 오류가 모이는 곳**을 하나 만드는 겁니다.

목표는 세 줄입니다. 서버는 **계속 살아있고**, 기록에는 **추적 가능한 상세**를 남기고, 사용자에게는 **안전한 일반 메시지**만 돌려준다 — 이게 §5 전체의 설계 원칙입니다.

터진 오류는 두 갈래로 흘러보낸다

에러 핸들링 흐름 — 예외가 서버를 죽이지 않게



01 한곳으로 모은다

엔드포인트마다 try/except를 다는 대신, **맨 아래에 그물 하나**를 칩니다. 어디서 난 오류든 미처 못 잡은 것은 전부 이 그물로 떨어져요.

02 기록으로 가는 길 — 전부

오류 종류·메시지·요청 경로·상세 추적까지 남깁니다. 내일 아침 "어젯밤 무슨 일이?"의 답이 여기 있어요 — 운영자를 위한 갈래입니다.

03 사용자로 가는 길 — 한 줄

응답은 **500** + "서버 내부 오류가 발생했습니다"뿐. 어느 파일 몇 번째 줄에서 터졌는지는 **절대** 내보내지 않아요 — 공격자에게 지도를 주지 않는 갈래입니다.

04 서버는 살아남는다

오류가 그물에서 **흡수**되므로 서버는 죽지 않습니다. 다음 요청은 아무 일 없었다는 듯 정상 처리돼요 — '죽지 않는 서버'의 핵심 장치입니다.

오류 → 한곳의 그물(전역 핸들러) → 기록(상세 전부) + 사용자(안전한 한 줄)로 갈라진다.

print를 졸업할 시간

새벽 3시에 서버가 이상했다 — `print("에러 발생")` 만 찍혀 있다면, 무엇을 알 수 있을까요? 거의 아무것도요.

디버깅 습관

print()

언제 일어났는지 모른다

`에러 발생` 한 줄엔 시각이 없습니다. 새벽 3시 장애인지 점심 해프닝인지, 시점을 짚을 수 없어요.

얼마나 심각한지 모른다

정상 기록과 치명적 오류가 똑같은 글씨로 섞입니다. 수천 줄에서 진짜 문제를 눈으로 골라내야 하죠.

끄거나 보낼 수 없다

운영에서 디버그 출력만 끄는 것도, 파일·외부 서비스로 보내는 것도 안 됩니다. 지우거나 다 보거나, 둘뿐이에요.

운영의 기본

logging

시간·출처가 자동으로

`2026-06-10 14:30 ERROR [ml_api] ...` 처럼 시각과 모듈명이 자동으로 붙습니다. 사고 시점과 위치가 바로 보여요.

심각도를 단계로 구분

INFO(정상)→WARNING(주의)→ERROR(오류)→CRITICAL(위험) 네 단계. "WARNING 이상만 보여줘"가 한 줄 설정입니다.

어디로든 흘려보낸다

설정만 바꾸면 같은 기록을 화면·파일·외부 모니터링으로 보냅니다. 부르는 코드는 그대로예요.

미들웨어 — 모든 요청이 지나는 단 하나의 길목

오류만 기록하면 반쪽입니다. "평소엔 몇 초였지?"를 모르면 "느려졌다"도 알 수 없으니까요. **미들웨어**는 모든 요청·응답이 반드시 지나는 통로 — 톨게이트처럼 들어올 때 시계를 켜고, 나갈 때 한 줄을 남깁니다.

미들웨어 — 모든 요청과 응답이 지나는 단 하나의 길목



app.add_middleware(...) 한 줄 — 이후 모든 엔드포인트의 호출이 자동으로 기록된다

01 들어올 때 — 시계 시작

핸들러가 실행되기 **전**에 시각을 찍습니다. 어떤 엔드포인트로 가는 요청이든 예외 없이 이 길을 지나요.

02 가운데 — 본 처리

요청을 핸들러(전처리→추론→응답)로 넘기고 결과를 돌려받습니다. 이 앞이 '처리 전', 뒤가 '처리 후'예요.

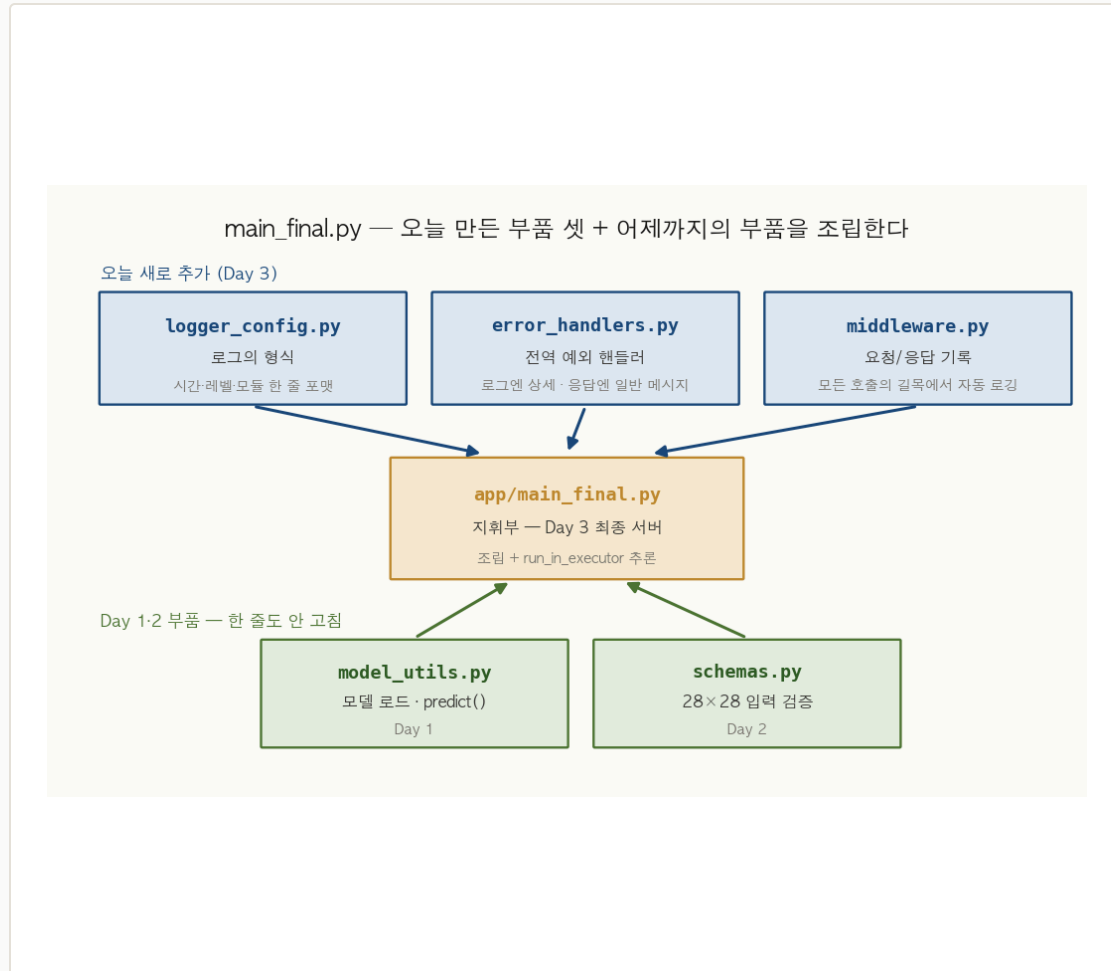
03 나갈 때 — 한 줄 기록

`POST /predict -> 200 (0.045s)` — 메서드·경로·상태·소요 시간이 자동으로 남습니다. 이 줄들이 쌓이면 트래픽 패턴과 성능 추세가 보여요.

04 한 번 등록, 전부 적용

한 번만 등록하면 이후 모든 엔드포인트에 적용됩니다. 전역 예외 처리와 같은 철학이죠 — **반복하지 말고, 길목에 한 번만.**

오늘 만든 부품 셋 + 어제까지의 부품



main_final.py가 지휘 — Day 1·2의 부품은 한 줄도 안 고치고 그대로 쫓는다.

01 오늘의 신상 부품 셋

로그 형식(logger), 전역 예외 처리(error handler), 요청 기록(middleware). 각자 **파일 하나·역할 하나** — 다른 프로젝트에 그대로 들고 가도 되는 범용 부품입니다.

02 지휘부 — main_final

부품들을 조립하고, 추론을 옆방(전용 풀)으로 위임하는 Day 3 최종 서버입니다. 조립이라고 해야 **부르는 줄 몇 개**가 전부예요.

03 Day 1·2 부품은 그대로

모델·추론 코드와 입력 검증은 **한 줄도 안 고칩니다**. 기능을 파일로 나눠 둔 덕에, 안정성 부품만 새로 끼우는 거예요.

04 분업의 효과

로그 형식을 바꾸려면 logger 파일만, 오류 정책을 바꾸려면 error 파일만 뽐니다. **파일 이름이 곧 책임의 경계**입니다.

정말 튼튼해졌는지, 직접 두드리려 본다

실제 MNIST 추론으로 동시 요청을 늘려 가며 — 속도와 생존, 둘 다 확인합니다.

STEP 01

헬스체크

`GET /health` 로 시작합니다.
모델이 올라왔는지 확인 —
이제 추론이 돌아도 이 응답은
즉시여야 합니다.



STEP 02

동시 요청 확대

실제 이미지로 1 → 2 → 4 → 8개를
동시에 던집니다. 함정 버전이라면
8개에 24초 — 개선 서버는 거의 안
늘어야 합니다.



STEP 03

추론 중 맥박

추론이 도는 와중에 `/health` 를
쫓러, §3의 '2.5초 지연'이
사라졌는지 — 0.0초
응답을 확인합니다.



STEP 04

비정상 입력

크기 틀린 픽셀(422), 깨진 이미지
(400)를 보내 적절한 상태 코드와
안전한 메시지가 오는지 봅니다. 내부
오류 내용이 보이면 불합격이에요.



STEP 05

생존과 흔적

비정상 요청 직후에도 정상 추론이 잘
되는지, 그리고 기록에
`WARNING ... -> 422` 같은
흔적이 남았는지 — 살아있음과 기록,
둘 다 합격해야 끝납니다.

한 줄 서던 서버가, 동시 요청에 건딘다

오늘 한 일

- ▶ **동기 vs 비동기** — 9초와 3초를 가른 건 '틈을 쓰는 전략'(그림 한 장)
- ▶ **이벤트 루프** — 일꾼이 한 명이라, 붙들리면 헬스체크까지 멈춘다
- ▶ **run_in_executor** — 무거운 추론만 옆방(전용 풀)으로 (CPU~코어수, GPU 1~2)
- ▶ **전역 예외 처리** — 기록엔 상세, 응답엔 안전한 한 줄, 서버는 생존
- ▶ **logging·미들웨어** — 모든 호출을 `메서드 경로 -> 상태 (시간)` 으로 자동 기록

비동기와 에러 핸들링은 '있으면 좋은 것'이 아니다 — 동시 사용자가 생기는 순간, 없으면 무너지는 토대다. 기다림은 겁치고, 계산은 떼어내고, 예외는 잡고, 모든 일은 기록하라.

다음 — DAY 4

- ▶ **Streamlit**으로 웹 UI — 코드 없이 클릭으로 쓰는 모델
- ▶ UI가 오늘의 FastAPI 서버를 불러 결과를 받는다
- ▶ 이미지 업로드 → 추론 → 결과 표시의 한 흐름
- ▶ Day 1의 '클라이언트-서버 분리' 그림의 완성
- ▶ 비개발자도 쓰는 모델 서비스로